



Workshop 2: The Down Server Dilemma (EC2 + VPC)

Core Focus: Designing reliable compute and network architecture using EC2 and VPC

Level: Beginner → Consultant Thinking

Duration: 90-120 minutes

Company Background: DataForge Analytics

DataForge Analytics is a consulting firm that builds dashboards for enterprise clients. They recently migrated their internal analytics platform to AWS using EC2, but the team lead keeps getting calls that “**the dashboard is down again.**”

When the server restarts, the public IP changes, and clients lose access to the reports they depend on.

Your job as a **Cloud Engineer in training**:

- ✓ Identify why this happens
- ✓ Design a reliable, business-ready fix
- ✓ Explain how your design helps the company save time and money

Learning Objectives

By the end of this workshop, you'll be able to:

- ✓ Launch EC2 instances inside a custom VPC
- ✓ Understand and configure Security Groups and route tables

- ✓ Diagnose networking issues and fix unreliable connectivity
- ✓ Use Elastic IPs for stable access
- ✓ Think like a consultant – tie reliability and scalability to business value

Part 1: Set Up the Networking Foundation

Why It Matters:

Before you ever deploy a server, you need a network that determines how your resources talk to each other and the internet.

Think of this as laying down roads before building houses.

Step-by-Step

- 1. Create a VPC**
 - Go to **VPC → Create VPC**
 - Name: **dataforge-vpc**
 - CIDR: **10.0.0.0/16**
 - Leave defaults → **Create**
- 2. Create a Subnet**
 - VPC: **dataforge-vpc**
 - Name: **public-subnet**
 - CIDR: **10.0.1.0/24**
 - Availability Zone: **us-east-1a**
 - **Create**
- 3. Attach an Internet Gateway**
 - Go to **Internet Gateways → Create** → dataforge-igw
 - Attach to **dataforge-vpc**
- 4. Set Route Table Rules**
 - Create a new Route Table → **public-rt**
 - Add Route → Destination 0.0.0.0/0 → **Target dataforge-igw**
 - Associate this route table with public-subnet

✓ **Result:** You've created a network where servers (EC2s) can connect to the internet safely.

🧠 **How Architects & Consultants Think**

"It's not just a network, it's an ecosystem."

An architect sees VPCs as **controlled environments** where every design choice affects scalability, cost, and security.

They ask:

- "Do we need separate subnets for public vs private resources?"
- "What happens if the AZ (Availability Zone) hosting this subnet fails?"
- "Should we implement multi-AZ design for fault tolerance?"
- "How do we enforce least privilege at the network level using Network ACLs?"

✓ **Architect takeaway:**

A consultant doesn't just connect resources, they design resilience.
Each component (VPC, subnet, gateway) should have a business justification.

👛 **Business Impact:**

- Building a custom VPC keeps client data isolated, satisfying compliance (HIPAA, PCI DSS, etc.).
- The subnet and routing design directly impacts availability and performance of the application.
- A well-architected network prevents cross-client exposure, reducing security risk and support costs.



Part 2: Launch and Configure Your EC2 Instance

Why It Matters

This is where your web application actually lives.

The **EC2 instance** is a virtual computer in the cloud, it's like renting a powerful laptop from Amazon that's always on (24/7), and accessible from anywhere.

You'll use it to host the company's analytics dashboard, just like DataForge would in real life.

Before You Start: What You're Doing

1. ☒ You're creating a cloud server
 - ☒ You'll connect it to the network you just built (your VPC)
 - ☒ You'll make sure it's publicly accessible but secure
 - ☒ You'll understand why each setting matters for business reliability
-

Step-by-Step: Launching Your EC2 Instance

1. **Go to EC2**
 - From the AWS Management Console, search for “**EC2**” at the top.
 - Click **EC2 (Elastic Compute Cloud)**.
You'll land on the **EC2 Dashboard**, which shows running instances and basic metrics.
2. Think of this dashboard as your **mission control** for all cloud servers.
3. **Click “Launch instance”**
 - You're about to create your first virtual machine in AWS.
 - A new window will open asking you to fill in some options.
We'll go through each one *slowly and intentionally*.
4. **Name your instance**
5. Under **Name and tags**, type:

- **dataforge-dashboard**
- This helps you identify this instance later, especially when you have multiple servers. In real companies, naming conventions matter because teams might manage hundreds of instances.

💡 **Tip: Consultants often name by purpose + environment:** dataforge-dashboard-prod or dataforge-dashboard-dev.

6. Choose an AMI (Amazon Machine Image)

You'll see a list of operating systems.

- Select:
Amazon Linux 2 (Free Tier Eligible)
👉 This is basically your operating system, like choosing Windows or macOS for a physical computer.
- Amazon Linux 2 is optimized for AWS and commonly used in production environments.

7. Choose an Instance Type

- Scroll down and choose:
t3.micro (or **t2.micro** if t3 is not free tier in your account).

💡 This defines the power of your cloud server, how many “brains” (CPU) and “memory” (RAM) it has. For learning, t3.micro is perfect: small, free-tier eligible, and fast enough for demos.

8. Network Settings

Expand this section, this is where most beginners get lost, so let's slow down:

- **VPC:** Choose your custom VPC (**dataforge-vpc**).
👉 This connects your server to the network you created earlier, not the default one. Think of this as picking which “office building” your server lives in.
- **Subnet:** Select public-subnet.
👉 This determines which “floor” of the building it's on, and this floor can talk to the internet.

- **Auto-assign Public IP:** Turn this **ON**.
 - ☞ This gives your server a public address, so you (and your users) can access it from anywhere.
 - Without it, it's like building a store but not giving it an address, no one could find it.

1. Configure Key Pair (Login Credentials)

Under **Key pair (login)**:

- If you already have one, select it.
- If not, click **Create new key pair**.
 - Name it: **dataforge-key**
 - Type: **RSA**
 - Format: **.pem**
 - Click **Create key pair**.
 - It will automatically download to your computer.

💡 **Keep this file safe.**

It's your private key, the only way to securely connect to your EC2 instance.
If you lose it, you can't log in.

Configure Security Group (Firewall Rules)

- Scroll to **Network settings → Firewall (Security Groups)**.
- Here's where you decide who can access your server and on what ports.
 - Choose **"Create security group"**
 - Name it: **dataforge-sg**
 - Add these rules:

Type	Protocol	Port	Source	Description
SSH	TCP	22	My IP	Allow secure admin access
HTTP	TCP	80	0.0.0.0/0	Allow website traffic

In simple terms:

- **SSH (22)** lets you log in as an engineer.
- **HTTP (80)** lets users see the website.
- The **“My IP”** restriction protects your server from strangers trying to log in and only allows your IP address to access the server.

Review and Launch

- Scroll down and click Launch instance.
- Wait a few seconds, then click View all instances.

Result:

- You’ll see your instance listed as running , green check mark 

Congratulations, you just deployed your first real cloud server!

Business Impact:

- Deploying the EC2 inside your own VPC ensures controlled access and accountability.
- Public IP access makes the service reachable to customers, but secure rules prevent unauthorized intrusion.
- Each configuration decision affects uptime and customer satisfaction, a direct link to revenue.

Checkpoint: Verify It’s Working

- Select your instance → check the **Public IPv4 address**.
- Copy that IP into your browser.
You should see a default AWS test page or blank page (which is okay, web server setup comes later).

How an Architect or Consultant Thinks

“I didn’t just launch a server, I deployed a business service.”

Here’s what separates a button-clicker from a cloud problem-solver:

Beginner Mindset	Architect / Consultant Mindset
"I just created a server."	"This instance must be part of a repeatable deployment process."
"I gave it a public IP."	"Should this service be behind a Load Balancer for high availability?"
"I added SSH and HTTP rules."	"In production, I'd use a Bastion host or VPN instead of opening SSH to the world."
"I'll use this for one app."	"Can we use this AMI as a template for auto-scaling or disaster recovery?"
"I'll monitor CPU later."	"Let's build CloudWatch alerts now to detect issues before users do."

✓ Architect takeaway:

A consultant doesn't just "launch", they **design for uptime, scale, and repeatability**. They ask: How can I automate this so the next engineer doesn't have to repeat my work?

💬 Reflection for Students

1. Why does giving the instance a Public IP make it accessible to clients?
2. What could happen if you opened **SSH (22)** to "Anywhere" instead of "My IP"?
3. How would you explain to a business owner why adding a Load Balancer or Elastic IP improves reliability?
4. Would you like me to write the **next section** ("Simulate and Diagnose the Problem") in this same style, explaining the IP change, why it breaks the site, and how architects design around that?

✓ **Result: You've successfully deployed a web server inside a secure network.**

🧠 How Architects & Consultants Think of what as done

“An instance isn’t just a server, it’s a resource in a lifecycle.”

Architects and consultants zoom out. They don’t just deploy, they **design for automation and recovery**.

They think:

- “How can we automate this setup using Terraform or CloudFormation?”
- “Should we use an Auto Scaling Group to handle sudden traffic spikes?”
- “How do we tag this instance for cost visibility across projects?”
- “If this were a client deployment, what SLAs (uptime guarantees) would we need?”

✅ **Consultant takeaway:**

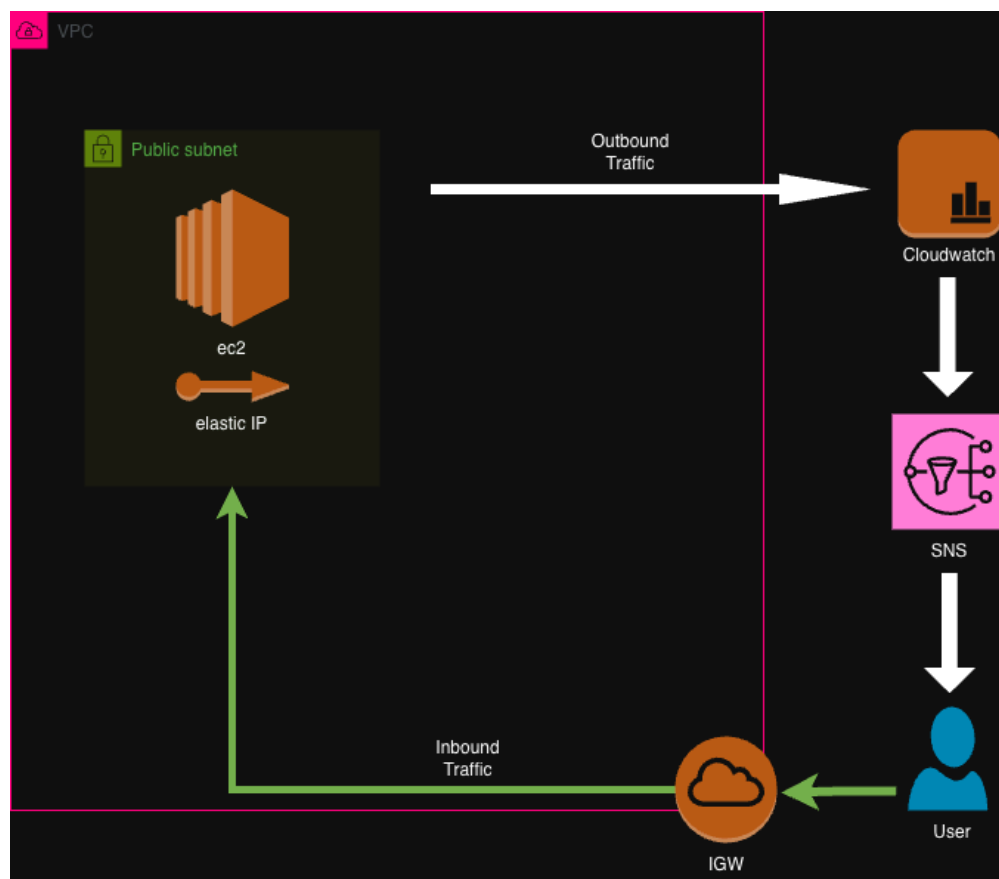
Every EC2 instance represents **a promise to the client**.

Design for consistency, not convenience.

This diagram shows where we’re headed.

Right now, we’re focused on the **green path**, getting users to our EC2 instance through the Internet Gateway.

Later, we’ll extend this design to include the white path, CloudWatch monitoring and SNS alerts, so our system doesn’t just run, it reports back when something’s off.



Part 3: Simulate and Diagnose the Problem

Scenario

The CEO says:

“Our dashboard keeps going down randomly. We can’t keep sending new IPs to clients.”

Symptoms:

- Pings intermittently fail.
- IP address changes after reboot.
- Logs show timeouts and failed connections.

Student Task

1. Stop and Start your EC2 instance.
2. Notice the **Public IP changed**.
3. Try reconnecting, it fails.
4. Identify the root cause: *The public IP isn’t persistent.*

How Architects & Consultants Think

“Downtime isn’t a tech problem, it’s a business cost.”

They ask:

- “How can we ensure predictable uptime for the client?”
- “What’s the real cost of these restarts (lost reports, delayed decisions)?”
- “Should we introduce Elastic Load Balancing or Route 53 DNS to decouple IPs?”

✓ **Architect takeaway:**

Architects design for reliability, not for luck.

They'd implement Elastic IPs or a load balancer to prevent future downtime.

Part 4: Implement the Fix

Step-by-Step

1. Go to **Elastic IPs → Allocate Elastic IP address**
2. Allocate → Associate with your EC2 instance
3. Test again, IP stays the same even after restart.

✓ **Result: You've stabilized the client's service.**

How Architects & Consultants Think

"Fixing the symptom isn't enough — we prevent it systemically."

They'd continue:

- "If we're using multiple servers, can we automate Elastic IP association?"
- "Can Route 53 DNS names point to these IPs for human-friendly access?"
- "How can CloudWatch alarms detect EC2 stops and auto-recover the instance?"

✓ **Consultant takeaway:**

True professionals turn **reactive fixes** into **proactive design principles**.

Business Impact:

- Using an Elastic IP eliminates downtime from IP changes, preserving client trust and contract SLA compliance.
- Predictable connectivity means fewer client tickets and fewer late-night emergency calls for engineers.
- For consulting teams, this reliability translates to recurring revenue and stronger client retention.

Part 5: Add Observability and Alerting

Step-by-Step

1. Go to **CloudWatch** → **Alarms** → **Create alarm**.
2. Metric: EC2 → CPUUtilization
3. Condition: > 80% for 5 minutes
4. Send notification via SNS topic (email).
5. Subscribe to the topic to receive alerts.

 **Result:** You now get real-time alerts when your server is under stress.

How Architects & Consultants Think

“Visibility equals control.”

They don’t just want alerts, they want **observability**.

They’d ask:

- “Should we use CloudWatch Dashboards for executive summaries?”
- “How do we aggregate logs with CloudWatch Logs or a SIEM tool?”
- “Can we integrate AWS Systems Manager for automated remediation?”

✓ Consultant takeaway:

Monitoring transforms you from a firefighter into a strategist.
It's not about reacting fast, it's about never being surprised.

Business Impact:

- Proactive monitoring detects issues before clients notice, turning your support team from reactive to strategic.
- Alerts help management make data-driven uptime reports for executive visibility.
- Every alert avoided = dollars saved in SLA penalties and downtime costs.

Final Reflection: Thinking Like a Consultant

Ask your students:

1. What was the root cause of DataForge's downtime?
2. How did we design a fix that improved both **technical reliability** and **business trust**?
3. What would you do differently if this were a **client contract** with uptime penalties?

Transformation Summary

You Started As...	You're Leaving As...
Someone who launches EC2 manually	Someone who designs cloud environments
Someone who reacts to downtime	Someone who prevents it strategically
Someone who thinks about technology	Someone who thinks about business outcomes

© 2025 Level Up In Tech, For enrolled members only.

Sharing outside this classroom is **prohibited** and **may result in account termination**. If you found this copy online, please report it to support@levelupintech.com with link and screenshot.